BIRZEIT UNIVERSITY

ENCS3340 - Artificial Intelligence Project Report

AI-Based Package Delivery Optimization Using Simulated Annealing and Genetic Algorithm

Student Name : Kareem Hamza

Student ID : 1220708

Instructor : Samah Al-Aydi

Section : 3

Date : 4/5/2025

Abstract

In the modern logistics and supply chain domain, optimizing the delivery of packages using computational methods has emerged as a crucial area of research. This project addresses the package delivery routing problem where a set of packages must be delivered by a fleet of vehicles with capacity constraints. Two well-known metaheuristic algorithms, Simulated Annealing (SA) and Genetic Algorithm (GA), were implemented to find near-optimal delivery routes that minimize the total travel distance. This system prompts user-defined packages and vehicle capacities and allows for interactive parameter tuning of both algorithms. Visualization modules demonstrate the effectiveness of the optimized routes. Comparative analysis reveals the strengths and limitations of each algorithm, guiding future improvements toward hybrid and real-time solutions.

Introduction

The increasing demand for efficient logistics has made route optimization a central concern in AI-driven delivery systems. Efficiently allocating packages to vehicles and determining optimal delivery sequences can significantly reduce operational costs and delivery times. Traditional algorithms often struggle with the complexity and variability of real-world delivery scenarios, making heuristic and metaheuristic methods more suitable.

Simulated Annealing and Genetic Algorithms are two such strategies capable of solving large combinatorial optimization problems. Simulated Annealing (SA) is inspired by the annealing process in metallurgy and explores the solution space by probabilistically accepting worse solutions to escape local minima. On the other hand, the Genetic Algorithm (GA) mimics natural evolution, utilizing crossover, mutation, and selection to evolve better solutions over generations.

This project applies these algorithms to solve a constrained delivery problem, considering package priorities and vehicle capacities. The program was implemented in Python, with support for user input, flexible parameter tuning, and real-time result visualization.

Problem Definition

The delivery optimization problem is defined as follows:

- There is a central depot (origin at coordinates (0,0)).
- A set of packages is defined, each with a weight, a delivery destination (x, y), and a priority level.
- A fleet of delivery vehicles, each with a specified weight capacity, must deliver these packages.
- Each package must be delivered by exactly one vehicle without exceeding its capacity.
- The objective is to minimize the **total distance travelled** by all vehicles, considering that they must return to the depot.

Constraints include:

- Each vehicle must not exceed its capacity.
- All packages must be assigned and delivered.
- The route should ideally reflect priority, though not enforced strictly due to algorithm simplicity.

Proposed Solution

The solution involves modelling the delivery problem as a combinatorial optimization task and applying both Simulated Annealing and Genetic Algorithm strategies.

Architecture Overview

The program is modular, including:

- Classes for Package and Vehicle
- Functions for user input and data validation
- Separate modules for SA and GA
- Visualization tools for displaying routes

Simulated Annealing (SA)

- **Initial Solution:** Packages are randomly assigned to vehicles respecting capacity.
- **Neighbour Generation:** A neighbour is created by swapping packages between vehicles.
- **Cost Function:** Sum of total distances of all vehicle routes.
- **Acceptance Probability:** Worse solutions may be accepted based on temperature.
- **Cooling Schedule:** Exponential decay of temperature.

Genetic Algorithm (GA)

- **Initial Population:** Random valid vehicle assignments.
- **Selection:** Tournament-based selection of parents.
- **Crossover:** Single-point crossover between vehicle routes.
- **Mutation:** Random reassignment of a package to another vehicle.
- **Fitness Function:** Inverse of total travel distance.

Main Control Flow

- The user chooses between SA and GA.
- Inputs are collected.
- The chosen algorithm is run and the final result is displayed and plotted.

Implementation Details

The program was implemented in Python and run in a local development environment using PyCharm.

Key Features:

- **Interactive Input:** Allows users to define custom problem sizes.
- **Data Structures:** Package and Vehicle classes encapsulate properties and behaviours.
- **Algorithms:** Encapsulated in functions with clean interfaces.
- **Visualization:** Matplotlib is used for plotting vehicle routes with coloured labels.

Parameter Customization:

- SA: Initial temperature, cooling rate, stopping temperature, iterations per temperature
- GA: Population size, mutation rate, number of generations

Error Handling:

- Capacity overflows are avoided during initialization.
- Invalid user inputs are handled gracefully.

Results and Discussion

The algorithms were tested with varying numbers of packages and vehicles.

Simulated Annealing Results:

- Generally converged to good solutions quickly.
- Susceptible to getting stuck in local optima if cooling is too fast.

Genetic Algorithm Results:

- Capable of discovering diverse solutions.
- Requires more iterations but often finds better global optima.

Visualization:

- SA and GA routes are plotted separately for clarity.
- Vehicle paths are color-coded, starting and ending at the depot.

Comparative Analysis:

- SA is simpler and faster for small instances.
- GA is more robust for larger and more complex scenarios.
- Both benefit from careful parameter tuning.

Conclusion and Future Work

This project demonstrates that both Simulated Annealing and Genetic Algorithms are viable for solving the delivery routing problem with constraints. Through modular design, interactive input, and rich visualization, the program offers a flexible experimentation environment.

Future Improvements:

- Incorporating time windows and delivery deadlines.
- Adding dynamic traffic conditions or real-time updates.
- Hybrid models combining SA and GA features.
- Using real-world map data (e.g., OpenStreetMap API).

References

1. Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671–680.
2. Holland, J. H. (1992). *Adaptation in Natural and Artificial Systems*. MIT Press.
3. Matplotlib Documentation. https://matplotlib.org/
4. Python Standard Library. https://docs.python.org/3/
5. ENCS3340 Course Materials.